

Chương 13: Tải và Tiền Xử Lý Dữ Liệu với TensorFlow

Giảng viên: [Tên Giảng Viên]

Đại Học Đông Á

Ngày 11 tháng 8 năm 2026

Nội dung Chương 13

1. Giới thiệu về API tf.data
2. Đọc và Tiền Xử Lý Dữ Liệu Lớn
3. TFRecord Format
4. Các Lớp Tiền Xử Lý của Keras
5. Tiền xử lý Biến Phân Loại (Categorical Features)
6. Embeddings (Nhúng)
7. TF Transform và TF Datasets
8. Tổng Kết

Vấn đề với dữ liệu cực lớn

- Trong Deep Learning, chúng ta thường làm việc với tập dữ liệu (dataset) khổng lồ không thể chứa vừa trong bộ nhớ RAM.
- Cần một cơ chế để **tải dữ liệu dần dần (lazy loading)** từ ổ cứng, tiền xử lý trên đường đi, và đưa từng batch vào GPU một cách trơn tru.
- TensorFlow cung cấp **Data API (tf.data)** để giải quyết hoàn hảo bài toán này.
- tf.data có thể đọc từ văn bản, tệp nhị phân, hoặc cơ sở dữ liệu SQL, đồng thời hỗ trợ nén và xử lý đa luồng.

Khởi tạo tf.data.Dataset từ Tensor

Cách đơn giản nhất để tạo Dataset là từ một Tensor có sẵn trên RAM:

```
import tensorflow as tf

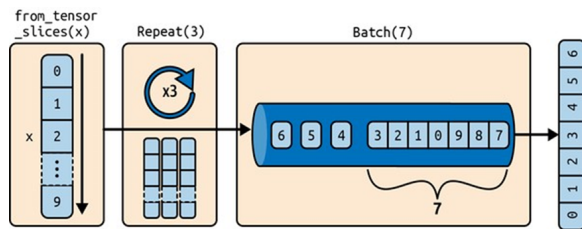
# Tao dataset tu mot tensor
X = tf.range(10) # 0, 1, 2, ..., 9
dataset = tf.data.Dataset.from_tensor_slices(X)

for item in dataset:
    print(item)
```

- Hàm `from_tensor_slices()` sẽ lấy tensor đầu vào và trả về một đối tượng Dataset.
- Mỗi phần tử của Dataset này là một phần tử của tensor ban đầu (slice dọc theo chiều thứ 0).

Các phép biến đổi chuỗi (Chaining Transformations)

- Sau khi có dataset, bạn có thể áp dụng các phép biến đổi lên nó bằng cách gọi các phương thức.
- Các phương thức này **không thay đổi dataset cũ** mà tạo ra một **dataset mới**.
- Các thao tác phổ biến: `repeat()`, `batch()`, `map()`, `filter()`.
- **Hình 13-1** mô tả một đường ống (pipeline) điển hình.



Hình 13-1: Xếp chuỗi các phép biến đổi tập dữ liệu

Ví dụ: Repeat và Batch

```
# Lap lai dataset 3 lan va nhom thanh cac batch kich thuoc 7
dataset = dataset.repeat(3).batch(7)

for item in dataset:
    print(item)
```

- Output:
- 'tf.Tensor([0 1 2 3 4 5 6], shape=(7,), dtype=int32)'
- 'tf.Tensor([7 8 9 0 1 2 3], shape=(7,), dtype=int32)'
- ... (Tổng cộng 30 phần tử, chia làm 5 batch).
- Lệnh `drop_remainder=True` trong `batch()` nếu muốn các batch phải bằng nhau.

Ví dụ: Map và Filter

- `map()`: Áp dụng một hàm lên mỗi phần tử của dataset. Thường dùng để tiền xử lý (vd: chuẩn hóa ảnh).
- `filter()`: Lọc bỏ các phần tử không thỏa mãn điều kiện.

```
# Nhân đôi giá trị
dataset = dataset.map(lambda x: x * 2)

# Lọc các phần tử < 10
dataset = dataset.filter(lambda x: x < 10)

# Lấy 3 phần tử đầu tiên
for item in dataset.take(3):
    print(item)
```

Trộn dữ liệu (Shuffling)

- Thuật toán Gradient Descent hoạt động tốt nhất khi các mẫu huấn luyện hoàn toàn độc lập và được phân phối ngẫu nhiên độc lập (IID).
- Hàm `shuffle()` sử dụng một **buffer** để chứa các phần tử, rồi bốc ngẫu nhiên từ buffer này.

```
dataset = tf.data.Dataset.range(10).repeat(3)
dataset = dataset.shuffle(buffer_size=5, seed=42).batch(7)
```

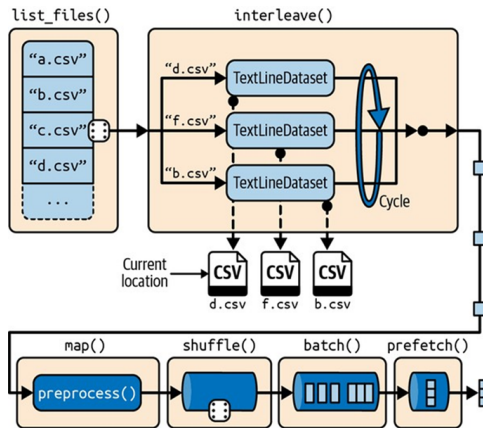
- **Lưu ý quan trọng:** `buffer_size` phải đủ lớn. Nếu `buffer_size=1`, dữ liệu sẽ không bị trộn chút nào! Nếu dữ liệu lớn, cần trộn ở cấp độ file trước.

Quy trình làm việc với nhiều file CSV

- Khi có hàng trăm GB dữ liệu chia thành hàng ngàn file CSV:

- 1 Trộn danh sách đường dẫn file (File paths).
- 2 Đọc luân phiên (Interleave) từ nhiều file cùng lúc.
- 3 Trộn dữ liệu ở cấp độ dòng (Buffer shuffle).
- 4 Phân lô (Batching).

- **Hình 13-2** mô phỏng quá trình cực kỳ mạnh mẽ này.



Hình 13-2: Tải

và tiền xử lý dữ liệu từ nhiều tệp CSV

Code: Interleave Đọc Song Song

```
filepath_dataset = tf.data.Dataset.list_files(train_filepaths, seed=42)

# interleave() sẽ đọc từ 5 file song song cùng một lúc
n_readers = 5
dataset = filepath_dataset.interleave(
    lambda filepath: tf.data.TextLineDataset(filepath).skip(1), # bỏ qua dòng Header
    cycle_length=n_readers
)

for line in dataset.take(3):
    print(line.numpy())
```

- Dữ liệu từ các file khác nhau sẽ được đan xen kẽ vào nhau.
- Tham số `num_parallel_calls=tf.data.AUTOTUNE` cho phép TF tự quyết định số luồng đọc.

Tiền Xử Lý Dữ Liệu Từng Dòng (Preprocessing)

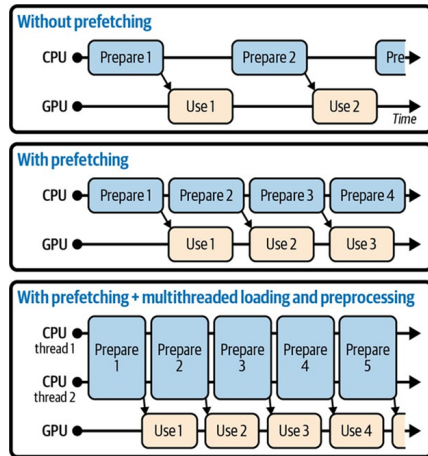
Chúng ta cần chuyển chuỗi (string) thành tensor số thực:

```
n_inputs = 8 # vd: dataset California Housing co 8 đặc trưng

def preprocess(line):
    # Định nghĩa kiểu dữ liệu mặc định cho mỗi cột
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    # Phân tích chuỗi CSV
    fields = tf.io.decode_csv(line, record_defaults=defs)
    x = tf.stack(fields[:-1]) # lấy 8 cột đầu làm X
    y = tf.stack(fields[-1:]) # cột cuối làm Y
    # Chuẩn hóa X (nếu đã tính sẵn mean, std)
    return (x - X_mean) / X_std, y
```

Quản lý Hiệu suất: Prefetching (Tải trước)

- Lệnh `dataset.prefetch(1)` vô cùng quan trọng.
- Trong khi mô hình (GPU) đang thực thi việc huấn luyện (forward/backward pass) ở Batch N.
- CPU sẽ tranh thủ làm việc để chuẩn bị sẵn Batch N+1 và đẩy vào bộ nhớ GPU.
- **Hình 13-3:** Thay vì CPU và GPU phải đợi nhau (như biểu đồ trên), chúng làm việc song song (biểu đồ dưới), tăng gấp đôi hiệu suất.



Hình 13-3:

Cơ chế Prefetching CPU và GPU hoạt động song song

Sử dụng Dataset trong Keras

- Keras hoàn toàn tương thích với `tf.data.Dataset`.
- Bạn không cần truyền X và y riêng lẻ, chỉ cần truyền toàn bộ dataset vào `fit()`.

Code: Huấn luyện với tf.data

```
# Xây dựng mô hình Keras
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="relu", input_shape=X_train.shape[1:]),
    keras.layers.Dense(1),
])
model.compile(loss="mse", optimizer="sgd")

# Huấn luyện
model.fit(train_dataset, steps_per_epoch=len(X_train) // batch_size,
          epochs=10, validation_data=valid_dataset)
```

Ví dụ Cụ Thể Về Các Giai Đoạn Đọc Dữ Liệu

- **Cấp độ 1:** Danh sách file. Gồm hàng ngàn file .csv chứa dữ liệu.
- **Cấp độ 2:** `interleave()`. Đọc 5 file cùng lúc. Các luồng đọc đan xen (interleave) dòng của 5 file này vào nhau.
- **Cấp độ 3:** `shuffle()`. Đưa các dòng vừa đọc được vào một bộ đệm (buffer). Chọn ngẫu nhiên từ bộ đệm này để đầu ra không bị lặp lại theo thứ tự file gốc.
- **Cấp độ 4:** `batch()`. Nhóm dữ liệu (vd: 32 dòng/batch).
- **Cấp độ 5:** `prefetch()`. Kéo trước batch tiếp theo.

Giải nén TFRecord (Parsing)

Khi đọc TFRecord, bạn nhận được chuỗi byte thô, cần giải nén bằng `tf.io.parse_single_example()`:

```
feature_description = {  
    "name": tf.io.FixedLenFeature([], tf.string, default_value=""),  
    "id": tf.io.FixedLenFeature([], tf.int64, default_value=0),  
    "emails": tf.io.VarLenFeature(tf.string),  
}  
  
def parse_record(record):  
    return tf.io.parse_single_example(record, feature_description)  
  
dataset = tf.data.TFRecordDataset(["my_data.tfrecord"]).map(parse_record)
```


Chuẩn Hóa Hình Ảnh (Image Standardization)

- Khi làm việc với hình ảnh, chúng ta cần thu phóng kích thước (Resize), cắt xén (Crop) và chuẩn hóa.
- Keras có thư viện tiền xử lý cực tốt: `tf.keras.layers.Resizing`, `tf.keras.layers.Rescaling`.
- Hoặc sử dụng `tf.image` API:
 - `tf.image.resize()`
 - `tf.image.random_crop()`
 - `tf.image.random_flip_left_right()`

Data Augmentation Bằng Keras Layers

```
data_augmentation = keras.models.Sequential([
    keras.layers.RandomFlip("horizontal"),
    keras.layers.RandomRotation(0.1),
    keras.layers.RandomZoom(0.2),
])
```

- Các lớp Tăng Cường Dữ Liệu (Data Augmentation) này **chỉ hoạt động trong giai đoạn huấn luyện (Training)**.
- Khi mô hình dự đoán thực tế (Testing / Inference), các lớp này sẽ tự động bị tắt!

Xử Lý Văn Bản: TextVectorization

- Tương tự StringLookup nhưng dành riêng cho đoạn văn bản nguyên vẹn.
- Nó bao gồm:
 - 1 Làm sạch (loại bỏ dấu câu, in thường).
 - 2 Tách từ (Tokenization), tách bằng khoảng trắng.
 - 3 Lập từ điển (Build vocabulary).
 - 4 Mã hóa mỗi từ thành một số nguyên (index).

Ví Dụ: TextVectorization

```
1 text_layer = keras.layers.TextVectorization()  
2 text_layer.adapt(["Hello world!", "Machine Learning is great"])  
3  
4 # Bien cau sau thanh danh sach cac con so  
5 print(text_layer(["Hello Machine"]))
```

- Đặc biệt, lớp này có thể chuyển đổi danh sách các số nguyên thành One-Hot hoặc TF-IDF.
- Rất hữu ích cho các mô hình NLP cơ bản.

Cơ chế Hashing (Băm Phân Loại)

- Đôi khi biến phân loại có Quá nhiều giá trị duy nhất (vd: 50,000 danh mục) và ta không muốn lập một từ điển khổng lồ.
- Hoặc ta liên tục có danh mục Mới (out of vocabulary).
- Kỹ thuật **Hashing (Băm)**: Đưa chuỗi vào một hàm băm, giả sử ép vào 1,000 nhóm (bins).
- Hai danh mục khác nhau có thể bị băm vào cùng một nhóm (Collisions), nhưng nhờ Nhúng (Embedding), mạng học vẫn tìm ra cách giải quyết.

Sử dụng HashingLayer trong Keras

```
hashing_layer = keras.layers.Hashing(num_bins=10)
# Ket qua: "Cuon sach A" -> bin 4, "Cuon sach B" -> bin 7, v.v.

encoded = hashing_layer(tf.constant(["Cuon sach A", "Cuon sach B"]))
```

- Không cần gọi `adapt()`. Không cần tạo từ điển!
- Cực kỳ nhanh, tốn rất ít bộ nhớ, có thể dùng on-the-fly.

- Thay vì tự huấn luyện một lớp Embedding cho các từ Tiếng Anh, tại sao không tải một lớp Embedding xuất sắc đã được Google huấn luyện trên hàng tỷ tài liệu?
- **TF Hub** là nền tảng chia sẻ các thành phần mạng nơ-ron có sẵn.
- Nó chứa hàng trăm mô hình nhúng văn bản (Text Embeddings) đa ngôn ngữ, mạng nhúng hình ảnh (Image Embeddings) từ ResNet.

Cách tải Mô Hình từ TF Hub

```
import tensorflow_hub as hub

hub_layer = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim50/2",
                           dtype=tf.string, input_shape=[],
                           trainable=True)

model = keras.Sequential([
    hub_layer,
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
```


Giới thiệu TFRecord

- Text CSV rất phổ biến, nhưng tốn dung lượng và phân tích cú pháp chậm.
- **TFRecord** là định dạng nhị phân nhẹ, linh hoạt, do TensorFlow thiết kế.
- Nó chứa một chuỗi các bản ghi (records) nhị phân có kích thước linh hoạt.
- TFRecord có thể được nén dễ dàng (GZIP) và luồng xử lý của tf.data hoạt động cực kì nhanh với chúng.
- Hữu ích nhất khi lưu trữ **Hình Ảnh** hoặc **Âm thanh** kèm theo nhãn.

Đọc và Viết TFRecord

```
# Viet
with tf.io.TFRecordWriter("my_data.tfrecord") as f:
    f.write(b"This is the first record")
    f.write(b"And this is the second record")

# Doc
filepaths = ["my_data.tfrecord"]
dataset = tf.data.TFRecordDataset(filepaths)
for item in dataset:
    print(item)
```

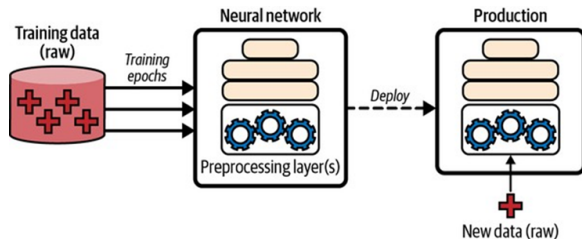
Protocol Buffers (Protobuf)

- TFRecord thường chứa các Protocol Buffers (protobuf) được tuần tự hóa (serialized).
- Protobuf là định dạng dữ liệu mã nguồn mở của Google, tương tự JSON nhưng nhẹ hơn và xử lý siêu nhanh.

```
# Định nghĩa cấu trúc protobuf (file .proto)
syntax = "proto3";
message Person {
    string name = 1;
    int32 id = 2;
    repeated string email = 3;
}
```

Tiền Xử Lý Bằng Keras Layers

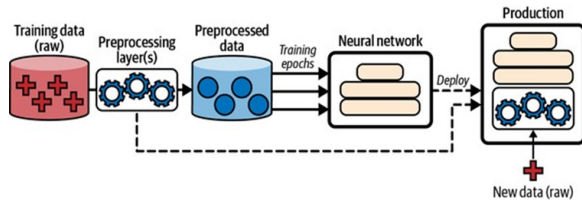
- Thay vì viết script tiền xử lý tách rời, bạn có thể **nhúng** thẳng quy trình tiền xử lý vào trong mô hình Keras!
- Lợi ích: Bất kì ai dùng model của bạn chỉ cần đẩy dữ liệu thô (chuỗi, ảnh gốc) vào, model tự xử lý rồi dự đoán.
- **Hình 13-4:** Mô hình tích hợp cả các lớp như Standardization, Text Vectorization ngay bên trong.



Hình 13-4: Bao gồm các lớp tiền xử lý bên trong một mô hình

Tiền Xử Lý Một Lần (AOT Preprocessing)

- Khuyết điểm của cách nhúng: Mỗi epoch đều phải tính lại Standardization → lãng phí.
- Tối ưu: Dùng tf.data để chạy lớp chuẩn hóa **một lần** toàn bộ dữ liệu (lưu ra cache).
- Khi xuất mô hình (Export) cho người dùng cuối, ta mới gắn lại lớp Tiền xử lý vào mô hình.
- **Hình 13-5** minh họa quy trình kép thông minh này.



Hình 13-5: Tiền xử lý dữ liệu một lần trước khi huấn luyện

Chuẩn Hóa Dữ Liệu (Normalization)

Keras cung cấp lớp Normalization thực hiện $x' = (x - \mu) / \sigma$.

```
norm_layer = keras.layers.Normalization()

# Tính toán (adapt) Mean và Variance trước trên tập Train
norm_layer.adapt(X_train)

# Áp dụng
X_train_scaled = norm_layer(X_train)

# Đưa vào model
model = keras.models.Sequential([
    norm_layer,
    keras.layers.Dense(1)
])
```

Rời Rạc Hóa Dữ Liệu Liên Tục (Discretization)

- Biến dữ liệu số thực thành dạng hạng mục (Categorical bins).
- Vd: Tuổi (24, 30, 45, 60) thành các nhóm (<25, 25-50, >50).

```
discretization_layer = keras.layers.Discretization(bin_boundaries=[25., 50.])
```

```
# 24 -> bin 0  
# 30, 45 -> bin 1  
# 60 -> bin 2
```

Mã Hóa One-Hot (One-Hot Encoding)

- Biến phân loại (ví dụ: Nghề nghiệp = "Kỹ sư", "Bác sĩ", "Giáo viên").
- Neural network không hiểu chuỗi văn bản.
- Mã hóa One-hot chuyển chúng thành vector nhị phân. "Kỹ sư- $[1, 0, 0]$. "Bác sĩ- $[0, 1, 0]$.

StringLookup và CategoryEncoding

- Trước hết, cần xây dựng từ điển (Vocabulary) ánh xạ chuỗi thành số nguyên (StringLookup).
- Sau đó, biến số nguyên thành One-Hot (CategoryEncoding).

```
vocab = ["<1H OCEAN", "INLAND", "NEAR OCEAN", "NEAR BAY", "ISLAND"]
# string_lookup se ánh xạ chuỗi vào các số 0, 1, 2...
lookup_layer = keras.layers.StringLookup(vocabulary=vocab)

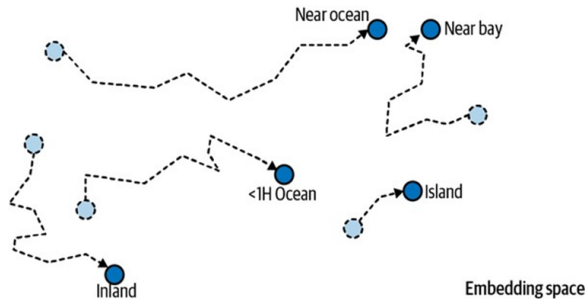
# encoded = CategoryEncoding
cat_encoding_layer = keras.layers.CategoryEncoding(num_tokens=lookup_layer.vocab_size())
```

Hạn Chế của One-Hot Encoding

- Nếu biến phân loại có quá nhiều giá trị (ví dụ: Tên 100,000 thành phố), mã hóa One-Hot sẽ tạo ra một vector khổng lồ dài 100,000 phần tử, toàn là số 0, chỉ có duy nhất một số 1.
- Lãng phí bộ nhớ khủng khiếp.
- Mạng nơ-ron phải nhân vô số trọng số dư thừa bằng 0.
- Giải pháp: **Embedding**.

Khái Niệm Nhúng (Embedding)

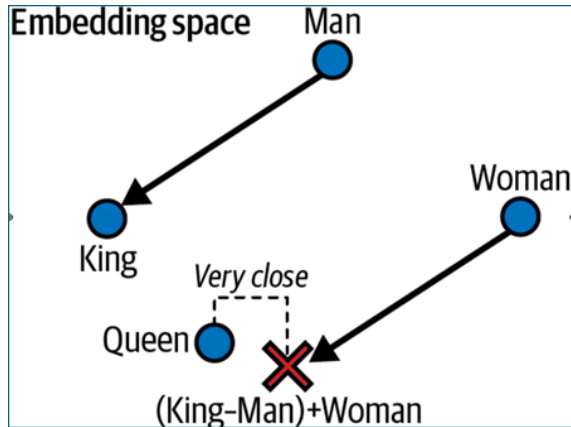
- Embedding là một vector nhỏ, liên tục (thường từ 10 đến 300 chiều) đại diện cho một danh mục.
- Thay vì mã hóa cố định (như One-hot), giá trị trong vector Embedding **có thể được học** và điều chỉnh qua quá trình backpropagation!
- **Hình 13-6** (trái): Bắt đầu khởi tạo ngẫu nhiên, các danh mục nằm rải rác.
- **Hình 13-6** (phải): Sau khi huấn luyện, các danh mục có ý nghĩa giống nhau tụ lại gần nhau ("NEAR BAY" và "NEAR OCEAN").



Hình 13-6: Quá trình tự động cải thiện
Embedding

Không Gian Ngữ Nghĩa của Word Embeddings

- Tính năng vĩ đại nhất của Nhúng Từ (Word Embedding).
- Mạng tự học cách mã hóa **khái niệm** vào không gian.
- **Hình 13-7:** Trong Word2Vec, khoảng cách (Man, Woman) gần như tương đương với (King, Queen).
- Phép tính đại số từ vựng:
 $King - Man + Woman \approx Queen$.



Hình 13-7: Trục Không gian ngữ nghĩa (giới tính)

Code: Xây Dựng Embedding trong Keras

```
1 vocab_size = 1000    # 1000 tu vung
2 embed_size = 2      # Vector gom 2 chieu (de de ve do thi)
3
4 # Lop embedding hoạt động như một bảng tra cứu
5 embed_layer = keras.layers.Embedding(input_dim=vocab_size, output_dim=embed_size)
6
7 model = keras.models.Sequential([
8     keras.layers.StringLookup(vocabulary=vocab),
9     embed_layer,
10    keras.layers.Dense(1)
11 ])
```

TensorFlow Transform (tf.Transform)

- Nhược điểm của việc dùng TF để tính Mean/Var cho tập Train là phải quét toàn bộ tập dữ liệu. Nếu dữ liệu quá lớn, việc này tốn rất nhiều thời gian.
- **TF Transform (tft)** ra đời cho việc Tiền xử lý quy mô cực lớn (ví dụ: chạy trên cụm Apache Spark, Apache Beam).
- Hàm tiền xử lý tft có thể chạy trên đám mây để chuẩn hóa hàng Terabyte dữ liệu.
- Sau đó tft sinh ra một TENSORFLOW GRAPH chứa sẵn công thức tiền xử lý (các hằng số mean, std) để nhúng vào mô hình khi Serving.

TensorFlow Datasets (TFDS)

- Google cung cấp kho dữ liệu khổng lồ chuẩn bị sẵn: **TensorFlow Datasets**.
- Bao gồm hàng trăm dataset nổi tiếng: MNIST, CIFAR10, IMDB, Wikipedia, ImageNet.

Sử dụng TFDS

```
import tensorflow_datasets as tfds

# Tu dong tai ve, tien xu ly va tra ve tf.data.Dataset
dataset = tfds.load(name="mnist", split="train")

# Truc tiep dua vao model
dataset = dataset.shuffle(10000).batch(32).prefetch(1)
for item in dataset:
    images = item["image"]
    labels = item["label"]
    # ...
```


Tổng Kết Chương 13

- **tf.data API:** Công cụ vô song để xây dựng đường ống dữ liệu (pipeline). Hỗ trợ đọc từ nhiều file song song (interleave), trộn (shuffle), và tăng tốc bằng prefetching.
- **TFRecord:** Định dạng nhị phân gọn nhẹ của TensorFlow.
- **Keras Preprocessing Layers:** Cung cấp cơ chế nhúng lớp tiền xử lý trực tiếp vào Mô hình (Normalization, StringLookup...).
- **Embeddings:** Kỹ thuật vĩ đại biểu diễn chuỗi/danh mục bằng vector số thực, tự động học ra ngữ nghĩa trong không gian liên tục (thay cho One-Hot Encoding lãng phí).